

BÁO CÁO ĐỒ ÁN MÔN HỌC

(Đồ án phát triển ứng dụng)

Lớp: IT003.Q21.CTTN

SINH VIÊN THỰC HIỆN

Mã sinh viên: 25521383

Họ và tên: Nguyễn Hải Phong

TÊN ĐỀ TÀI: Cycle of Curse, Top Down Bosses Rush Game

CÁC NỘI DUNG CẦN BÁO CÁO:

1. Giới thiệu đồ án:

a. Mô tả chung về ứng dụng:

- Đồ án xây dựng một trò chơi hành động nhập vai (Action RPG) thể loại Boss Rush, lấy cảm hứng từ lối chơi của *Soul Knight* và cốt truyện thế giới chú thuật của *Jujutsu Kaisen*.
- Cốt truyện: Người chơi nhập vai một Chú thuật sư bị nhốt trong Mê cung. Nhiệm vụ duy nhất là chiến đấu và tiêu diệt các Chú linh cấp cao (Bosses) để phá vỡ kết giới và thoát ra ngoài.
- Hệ thống chiến đấu: Nhân vật sở hữu các bộ kỹ năng đặc thù (Thuật thức), sử dụng năng lượng "Chú lực" để tấn công và thực hiện các kỹ năng né đòn (Dash) ở tốc độ cao.
- Đồ họa 2.5D & Camera: Sử dụng kỹ thuật Y-Sorting để quản lý chiều sâu hình ảnh, giúp nhân vật tương tác tự nhiên với môi trường (đứng trước/sau vật thể). Camera thông minh có khả năng tự động bám đuôi (Tracking) và chặn biên (Clamping) để đảm bảo trải nghiệm nhìn không bị gián đoạn.
- Cơ chế Va chạm (Physics): Tối ưu hóa trải nghiệm bằng cách tách biệt Hitbox (vùng nhận sát thương/va chạm) và Rect (khung hình hiển thị), giúp các pha né đòn và di chuyển trở nên chính xác, mượt mà hơn.
- Trò chơi xây dựng hệ thống đa bản đồ với cơ chế chuyển tiếp (Transition) liền mạch, cho phép người chơi di chuyển giữa các khu vực lãnh địa khác nhau sau khi hoàn thành nhiệm vụ.
- Cơ chế Animation và Âm thanh: Mỗi loại kỹ năng, nhân vật và quái vật đều được thiết kế bộ hoạt ảnh (Animation) riêng biệt để thể hiện trạng thái chiến đấu. Đi kèm với đó là hệ thống âm thanh đặc trưng cho từng loại thuật thức và hành động của quái vật, giúp tăng tính phản hồi và trải nghiệm thính giác sống động.

b. Các CTDL và giải thuật đã được sử dụng: (cần làm rõ các câu hỏi: what, why)

Các thuật toán và CTDL đã áp dụng:

- Ma trận 2 chiều (2D Array): Dùng để thiết kế và quản lý dữ liệu cho hệ thống bản đồ, giúp truy xuất địa hình và xử lý va chạm với độ phức tạp $O(1)$.
- Thuật toán Sắp xếp (Sorting): Xử lý thứ tự hiển thị (Z-index) dựa trên tọa độ trục Y của các thực thể, đảm bảo tính đúng đắn về mặt không gian trong môi trường 2.5D.
- Thuật toán Loang (Flood Fill): Ứng dụng trong giai đoạn tiền xử lý tài nguyên. Thuật toán tự động nhận diện và lọc bỏ vùng nền (background) của các nhân vật từ Sprite Sheet thô, giúp tối ưu hóa việc tạo các ảnh nhân vật có độ trong suốt chính xác.

Các thuật toán dự kiến bổ sung:

- Tìm kiếm theo chiều rộng (BFS - Auto Aim): Xây dựng cơ chế tự động ngắm bắn (Auto Aim). Thuật toán sẽ quét lưới tọa độ xung quanh nhân vật để xác định vị trí kẻ địch gần nhất, giúp tự động định hướng chiều thức một cách hiệu quả.
 - Cấu trúc dữ liệu Đống (Binary Heap): Dự kiến sử dụng để kiểm soát hệ thống âm thanh và thứ tự ưu tiên các mục tiêu của cơ chế Auto Aim. Sử dụng Min-Heap giúp hệ thống luôn ưu tiên xử lý các âm thanh hoặc mục tiêu quan trọng nhất dựa trên khoảng cách, tối ưu hóa hiệu suất CPU.
 - *Thuật toán A (A-Star)*: * Nâng cấp trí tuệ nhân tạo cho Boss và quái vật (như con chó ngọc), giúp chúng di chuyển thông minh, biết né vật cản để áp sát người chơi thay vì chỉ đi đường thẳng đơn giản.
2. Quá trình thực hiện

Tuần 1: Nghiên cứu kỹ thuật và Thiết lập nền tảng

- **Nghiên cứu công nghệ:** Tìm hiểu chuyên sâu về thư viện Pygame, các cơ chế xử lý Game Loop và quản lý tài nguyên hệ thống.
- **Khởi tạo dự án:** Thiết lập môi trường phát triển, cài đặt và cấu hình Git để kiểm soát phiên bản (Version Control), đảm bảo quy trình quản lý mã nguồn khoa học.
- **Phát triển kịch bản và Thiết kế hệ thống:** Xây dựng cốt truyện "Cycle of Curses", phác thảo danh sách các tính năng cốt lõi (To-do list) và phân tích các bài toán thuật toán cần giải quyết.
- **Xây dựng tài nguyên đồ họa:**
 - **Tìm kiếm tư liệu:** Nghiên cứu và thu thập các hình ảnh nhân vật, quái vật và bối cảnh từ các nguồn tài nguyên mở trên mạng. Các nhân vật này thường được lưu dưới dạng một tấm ảnh lớn chứa nhiều tư thế di chuyển, tấn công khác nhau (thường gọi là Sprite Sheet).
 - **Tiền xử lý hình ảnh:** Sử dụng thuật toán Flood Fill để tự động nhận diện vùng màu nền bao quanh nhân vật. Thuật toán sẽ "loang" ra để tách biệt phần nhân vật chính ra khỏi các vùng màu dư thừa, từ đó cắt nhỏ tấm ảnh lớn thành từng khung hình đơn lẻ có nền trong suốt, phục vụ cho việc tạo chuyển động (Animation) trong game.

Tuần 2: Xây dựng cơ chế lõi và quản lý không gian

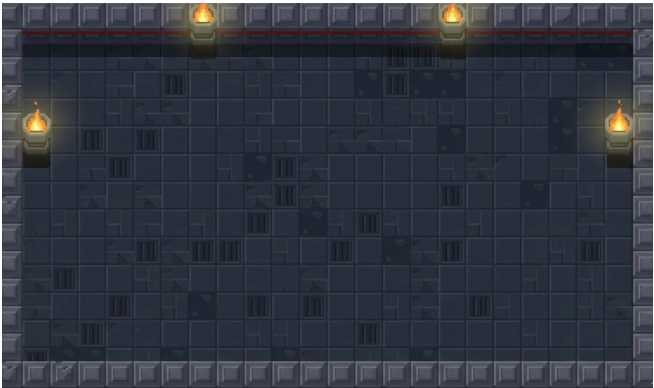
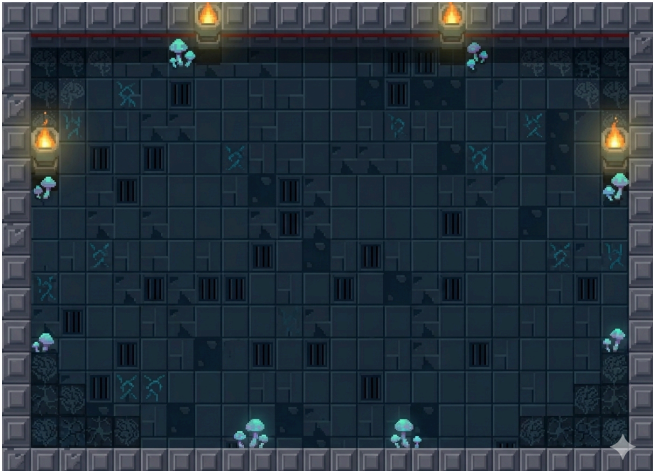
- Thiết kế hệ thống Map: Xây dựng ma trận bản đồ (Grid Matrix) cho nhiều khu vực khác nhau. Thiết lập cơ chế render địa hình dựa trên cấu trúc Mảng 2 chiều để tối ưu hóa truy xuất.
- Hệ thống Camera và Hiển thị: Phát triển cơ chế Camera Tracking tự động bám đuổi nhân vật và xử lý thuật toán Sorting (Y-Sort) để quản lý chiều sâu lớp vẽ (Z-index) trong không gian 2.5D.
- Cơ chế vận động và Va chạm: Lập trình hệ thống di chuyển (Movement) và xử lý va chạm (Collision) dựa trên việc tách biệt giữa hình ảnh hiển thị và Hitbox thực tế.
- Phát triển hoạt ảnh (Animation): Xây dựng máy trạng thái (State Machine) để quản lý các bộ Animation cho Player và Boss (các đợt Round 2, 3), bao gồm các trạng thái di chuyển và Dash.

3. Kết quả đạt được

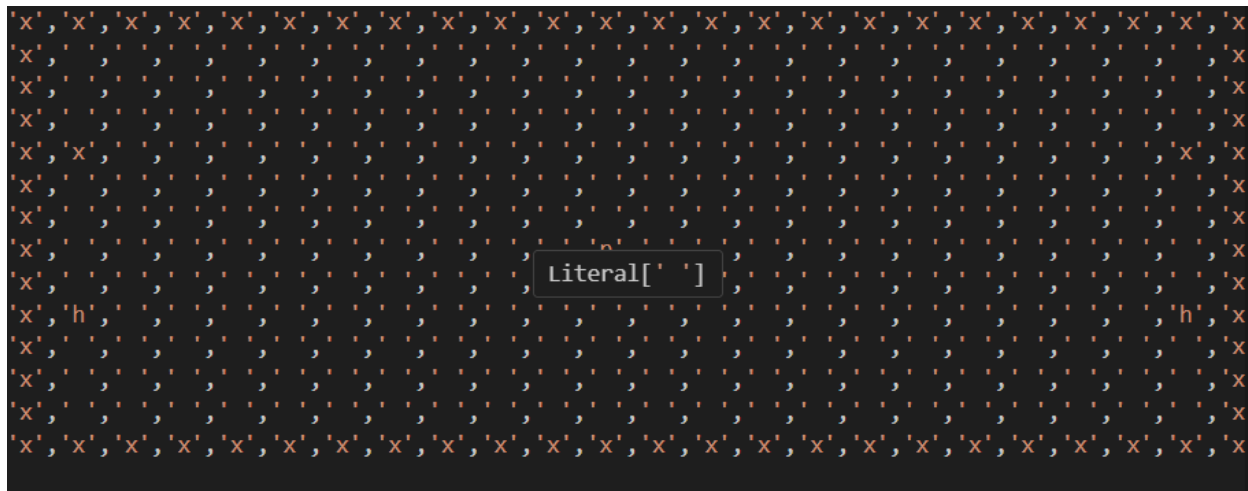
Hình ảnh các nhân vật, background:

Tên hình ảnh	Hình ảnh
Nhân vật chính	
Boss round 2 (Megumi)	

Boss round 2 (Megumi with skill)	
Boss round 3 (Sukuna)	
Quái nhỏ (Chó sói)	
Background 1	

Background 2	
Weapon 1	
Background 3	

Cơ chế World Map để kiểm soát collision:



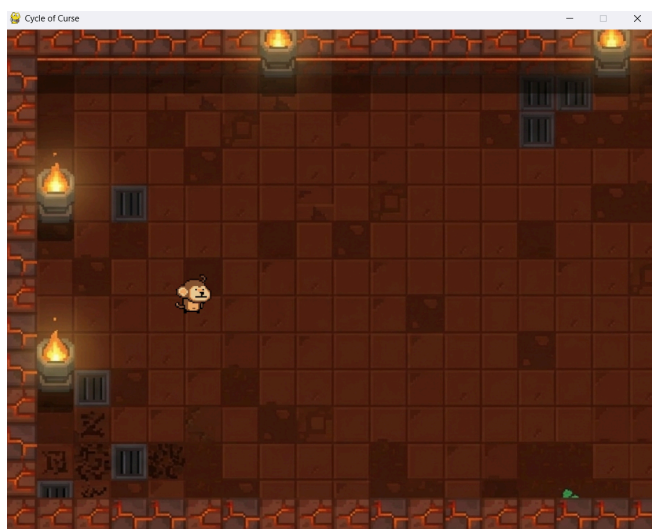
Settings cho game:

```

WIDTH = 900
HEIGHT = 700
FPS = 60
COLS = 24
ROWS = 14
T_WIDTH = 1224 // COLS
T_HEIGHT = 711 // ROWS
# PLAYER_INDEX: 1=Monkey, 2=Megumi, 3=Sukuna
PLAYER_INDEX = 2

```

Minh họa góc nhìn player trong game:



Tuần 3 Xây dựng cơ chế lỗi và quản lý không gian

- Xây dựng bộ vũ khí cho nhân vật

Tên vũ khí	Hình ảnh
Kiếm	
Rìu	
Thương	
Sai	
Kiếm một cạnh	
Bình máu	

- Xây dựng bộ stat riêng cho từng vũ khí

```

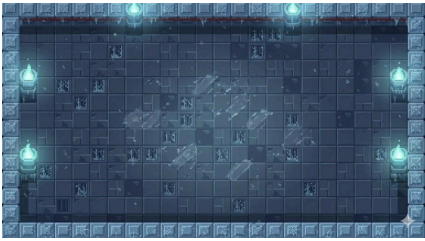
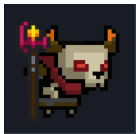
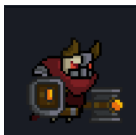
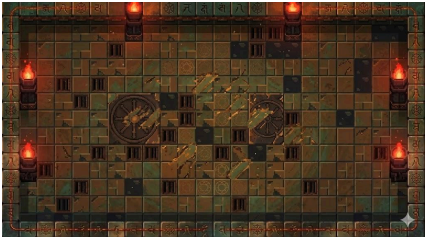
weapon_data = {
    'sword': {'cooldown': 100, 'damage': 15, 'graphic': '../graphics/weapons/sword/full.png'},
    'lance': {'cooldown': 400, 'damage': 30, 'graphic': '../graphics/weapons/lance/full.png'},
    'axe': {'cooldown': 300, 'damage': 20, 'graphic': '../graphics/weapons/axe/full.png'},
    'rapier': {'cooldown': 50, 'damage': 8, 'graphic': '../graphics/weapons/rapier/full.png'},
    'sai': {'cooldown': 80, 'damage': 10, 'graphic': '../graphics/weapons/sai/full.png'}}

# magic
magic_data = {
    'heal': {'strength': 20, 'cost': 0, 'graphic': '../image/potion.png'}}

```

- Xây dựng UI cho thanh máu, mana, hiển thị map, chuyển đổi vũ khí
- Xây dựng cơ chế di chuyển quanh map theo camera

- Xây dựng hiệu ứng tấn công cho nhân vật
- Hoàn thiện hình ảnh các map chiến đấu cho từng boss, quái nhỏ, quái phụ trong từng map đó

Map 1		 
Map 2		   
Map 3		    
Map 4		

3. Kết quả đạt được

Link github: Link code github: https://github.com/sherwyndd/DSA_GAME_CycleOfCurse

- Hoàn thiện bộ map
- Hoàn thiện một phần bộ kỹ năng nhân vật
- Định hình cơ chế tấn công
- Gắn hoàn thiện hình ảnh cho các quái vật
- Hoàn thiện bộ UI cho thanh máu, mana, chuyển đổi map
- Tiến trình 60/100

4. Tài liệu tham khảo

Video hướng dẫn dạng Top down game: <https://www.youtube.com/watch?v=QU1pPzEGrQw&t=8404s>

Hình ảnh tham khảo cho nhân vật

https://www.reddit.com/r/SoulKnight/comments/1ak8ih1/soul_knight_x_jjk/

https://www.reddit.com/r/SoulKnight/comments/1ic349m/soul_knight_x_jjk_part2/

5. Phụ lục 1: Giới thiệu (demo) kết quả

Video demo Tuần 1 + 2:

<https://drive.google.com/file/d/1OUXleghaCEY8jYdGriZw-A0IM3P1AaYa/view?usp=sharing>

Video demo Tuần 3:

<https://drive.google.com/file/d/1-0ZFYJiisSQSiJCEde05Ruq6K8vyaqKU/view?usp=sharing>

6. PHỤ LỤC 2: TÀI LIỆU DOCSTRING CỦA DỰ ÁN CYCLE OF CURSE

```
=====
MODULE: PLAYER (Nhân vật)
=====
```

CLASS: Player

Mô tả: Lớp đại diện cho nhân vật người chơi trong game.

Quản lý di chuyển, va chạm, hoạt ảnh (animation) và các kỹ năng của nhân vật.

Method: `__init__()`

Chi tiết: Khởi tạo đối tượng Player.

Args:

pos (tuple): Vị trí ban đầu của nhân vật (x, y).

groups (list): Danh sách các nhóm sprite mà nhân vật thuộc về.

obstacle_sprites (pygame.sprite.Group): Nhóm các sprite vật cản để kiểm tra va chạm.

Method: `__repr__()`

Chi tiết: Return `repr(self)`.

Method: `add()`

Chi tiết: add the sprite to groups

`Sprite.add(*groups)`: return None

Any number of Group instances can be passed as arguments. The Sprite will be added to the Groups it is not already a member of.

Method: `add_internal()`

Chi tiết: For adding this sprite to a group internally.

:param group: The group we are adding to.

Method: `alive()`

Chi tiết: does the sprite belong to any groups

`Sprite.alive()`: return bool

Returns True when the Sprite belongs to one or more Groups.

Method: `groups()`

Chi tiết: list of Groups that contain this Sprite

`Sprite.groups()`: return group_list

Returns a list of all the Groups that contain this Sprite.

Method: input()

Chi tiết: Xử lý các phím nhấn từ người dùng để điều khiển nhân vật.

- W, A, S, D: Di chuyển.
- SPACE: Tấn công (Attack).
- L-CTRL: Lướt nhanh (Dash).

Method: kill()

Chi tiết: remove the Sprite from all Groups

Sprite.kill(): return None

The Sprite is removed from all the Groups that contain it. This won't change anything about the state of the Sprite. It is possible to continue to use the Sprite after this method has been called, including adding it to Groups.

Method: remove()

Chi tiết: remove the sprite from groups

Sprite.remove(*groups): return None

Any number of Group instances can be passed as arguments. The Sprite will be removed from the Groups it is currently a member of.

Method: remove_background_floodfill()

Chi tiết: Thuật toán Flood Fill để xóa nền ảnh một cách thông minh.

Chỉ xóa các điểm ảnh nền kết nối với các góc, giúp giữ lại các chi tiết trắng bên trong (như mắt).

Args:

surf (pygame.Surface): Surface cần xử lý xóa nền.

Returns:

pygame.Surface: Surface đã được làm sạch nền.

Method: `remove_internal()`

Chi tiết: For removing this sprite from a group internally.

:param group: The group we are removing from.

Method: `update()`

Chi tiết: method to control sprite behavior

`Sprite.update(*args, **kwargs)`:

The default implementation of this method does nothing; it's just a convenient "hook" that you can override. This method is called by `Group.update()` with whatever arguments you give it.

There is no need to use this method if not using the convenience method by the same name in the `Group` class.

=====

MODULE: LEVEL (Màn chơi & Camera)

=====

CLASS: `Level`

Mô tả: Lớp quản lý cấp độ (level) của trò chơi.

Chịu trách nhiệm tạo bản đồ, quản lý các nhóm sprite và điều phối quá trình vẽ/cập nhật.

Method: `__init__()`

Chi tiết: Initialize self. See `help(type(self))` for accurate signature.

Method: `create_map()`

Chi tiết: Đọc dữ liệu từ bản đồ (`WORLD_MAP`) và khởi tạo các đối tượng tương ứng (Tiles, Player).

Sử dụng kỹ thuật Crop tự động từ ảnh nền để tạo các vật cản.

CLASS: YSortCameraGroup

Mô tả: Lớp nhóm Sprite tùy chỉnh tích hợp Camera và sắp xếp theo trục Y.

Giúp tạo hiệu ứng chiều sâu (đối tượng ở dưới sẽ che đối tượng ở trên) và di chuyển camera theo người chơi.

Method: `__init__()`

Chi tiết: Initialize self. See `help(type(self))` for accurate signature.

Method: `__len__()`

Chi tiết: return number of sprites in group

`Group.len(group)`: return int

Returns the number of sprites contained in the group.

Method: `__repr__()`

Chi tiết: Return `repr(self)`.

Method: `add()`

Chi tiết: add sprite(s) to group

`Group.add(sprite, list, group, ...)`: return None

Adds a sprite or sequence of sprites to a group.

Method: `add_internal()`

Chi tiết: For adding a sprite to this group internally.

:param sprite: The sprite we are adding.

:param layer: the layer to add to, if the group type supports layers

Method: `clear()`

Chi tiết: erase the previous position of all sprites

`Group.clear(surface, bgd)`: return `None`

Clears the area under every drawn sprite in the group. The `bgd` argument should be `Surface` which is the same dimensions as the screen surface. The `bgd` could also be a function which accepts the given surface and the area to be cleared as arguments.

Method: `copy()`

Chi tiết: copy a group with all the same sprites

`Group.copy()`: return `Group`

Returns a copy of the group that is an instance of the same class and has the same sprites in it.

Method: `custom_draw()`

Chi tiết: Vẽ các sprite lên màn hình với sự tính toán offset của camera và sắp xếp thứ tự hiển thị.

Args:

player (`Player`): Đối tượng player để camera đi theo.

Method: `draw()`

Chi tiết: draw all sprites onto the surface

`Group.draw(surface, special_flags=0)`: return `Rect_list`

Draws all of the member sprites onto the given surface.

Method: `empty()`

Chi tiết: remove all sprites

`Group.empty()`: return None

Removes all the sprites from the group.

Method: `has()`

Chi tiết: ask if group has a sprite or sprites

`Group.has(sprite or group, ...)`: return bool

Returns True if the given sprite or sprites are contained in the group. Alternatively, you can get the same information using the 'in' operator, e.g. 'sprite in group', 'subgroup in group'.

Method: `has_internal()`

Chi tiết: For checking if a sprite is in this group internally.

:param sprite: The sprite we are checking.

Method: `remove()`

Chi tiết: remove sprite(s) from group

`Group.remove(sprite, list, or group, ...)`: return None

Removes a sprite or sequence of sprites from a group.

Method: `remove_internal()`

Chi tiết: For removing a sprite from this group internally.

:param sprite: The sprite we are removing.

Method: `sprites()`

Chi tiết: get a list of sprites in the group

Group.sprites(): return list

Returns an object that can be looped over with a 'for' loop. (For now, it is always a list, but this could change in a future version of pygame.) Alternatively, you can get the same information by iterating directly over the sprite group, e.g. 'for sprite in group'.

Method: update()

Chi tiết: call the update method of every member sprite

Group.update(*args, **kwargs): return None

Calls the update method of every member sprite. All arguments that were passed to this method are passed to the Sprite update function.

=====

MODULE: MAIN (Khởi chạy hệ thống)

=====

CLASS: Game

Mô tả: Lớp khởi tạo chính của trò chơi.

Thiết lập cửa sổ, vòng lặp game (game loop) và quản lý tiến trình chung của ứng dụng.

Method: __init__()

Chi tiết: Initialize self. See help(type(self)) for accurate signature.

Method: run()

Chi tiết: Vòng lặp chính của trò chơi.

Xử lý các sự kiện, cập nhật logic và vẽ lại khung hình (60 FPS).

